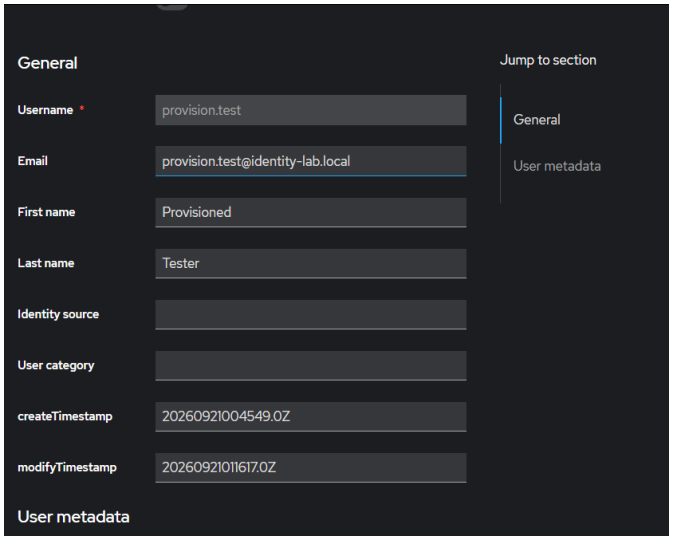
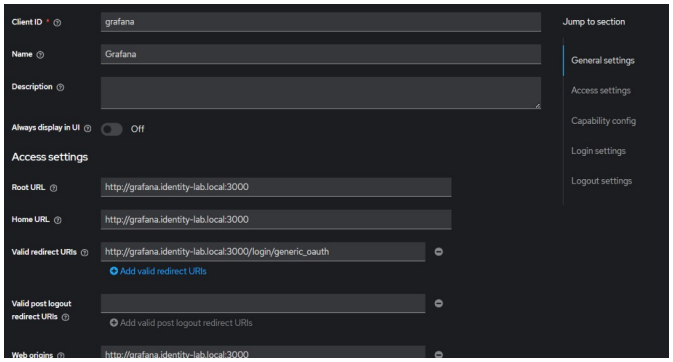
1. Test User Profile

I verified that the `provision.test` account was enabled and had a populated email address. This profile information is required for signing in to Grafana through Keycloak.



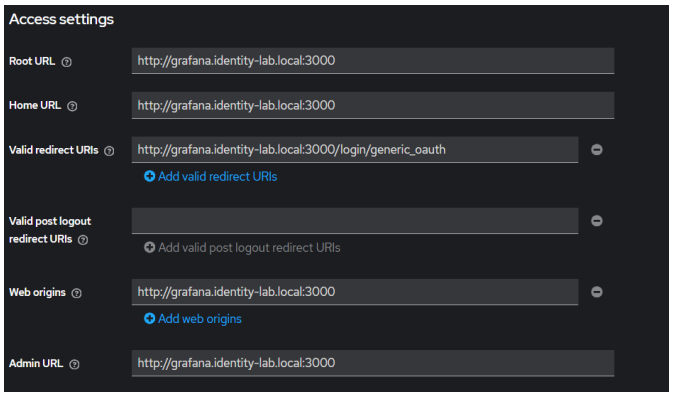
2. OIDC Client Configuration

I created the `grafana` OpenID Connect client with client authentication and Standard Flow enabled. Direct Access Grants were disabled because the integration uses Authorization Code Flow.



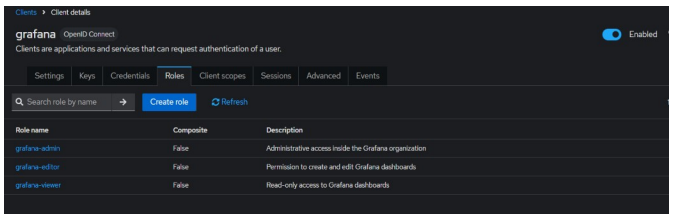
3. Redirect URI Configuration

I configured the Grafana application URLs and set the exact redirect URI to `http://grafana.identity-lab.local:3000/login/generic_oauth`. This allows Keycloak to return the authorization response to Grafana.



4. Grafana Client Roles

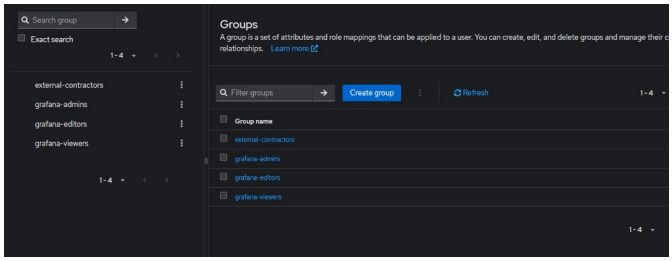
I created three client roles: `grafana-viewer`, `grafana-editor`, and `grafana-admin`. These roles represent the application's three access levels.



5. Grafana Access Groups

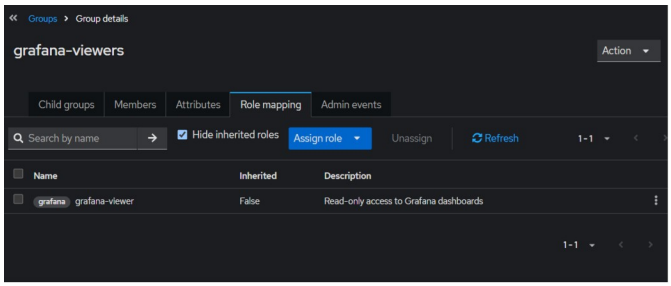
I created the `grafana-viewers`, `grafana-editors`, and `grafana-admins` groups in Keycloak. These groups provide centralized management of Grafana

access.



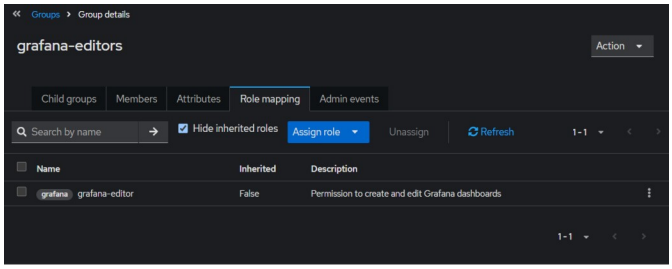
6. Viewer Group Role Mapping

I assigned the `grafana-viewer` client role to the `grafana-viewers` group. Members of this group inherit the role used to grant Viewer access in Grafana.



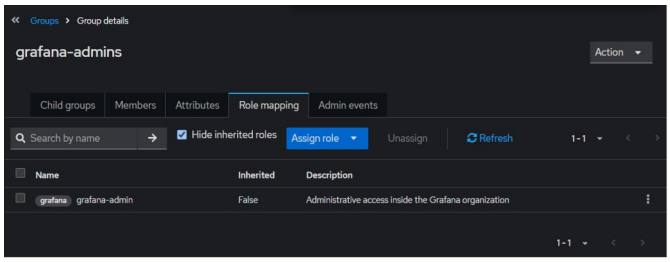
7. Editor Group Role Mapping

I assigned the `grafana-editor` client role to the `grafana-editors` group. Members inherit the role used to grant dashboard creation and editing permissions.



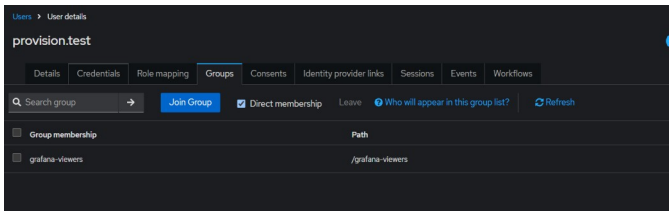
8. Admin Group Role Mapping

I assigned the `grafana-admin` client role to the `grafana-admins` group. This role maps to administrative access within the Grafana organization.



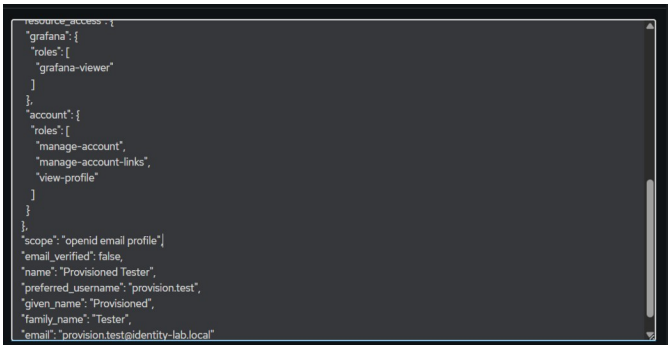
9. User Membership in the Viewer Group

I added `provision.test` to the `grafana-viewers` group. The user received the Viewer client role through group membership rather than a direct role assignment.



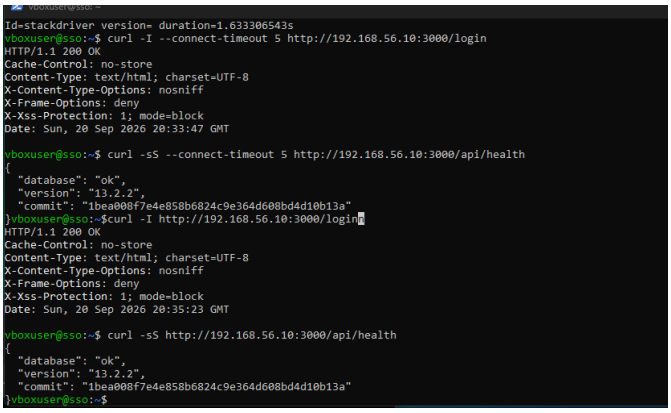
10. Viewer Role in the Access Token

I used Keycloak's Evaluate feature to inspect a generated access token for `provision.test`. The `grafana-viewer` role appeared under `resource_access.grafana.roles`, confirming the group-based role mapping.



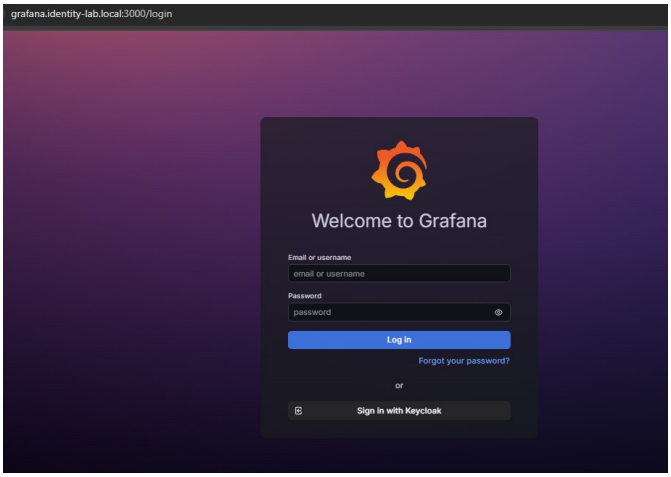
11. Grafana Container Deployment

I deployed Grafana in Docker with persistent storage and published port 3000 on `192.168.56.10`. The running container and successful HTTP response confirmed that Grafana was available.



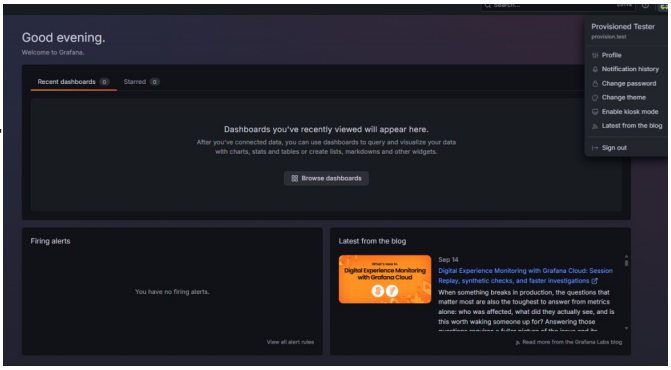
12. Keycloak Login Option

I opened the Grafana login page and verified that the "Sign in with Keycloak" button was displayed. This confirmed that the Generic OAuth login option was enabled.



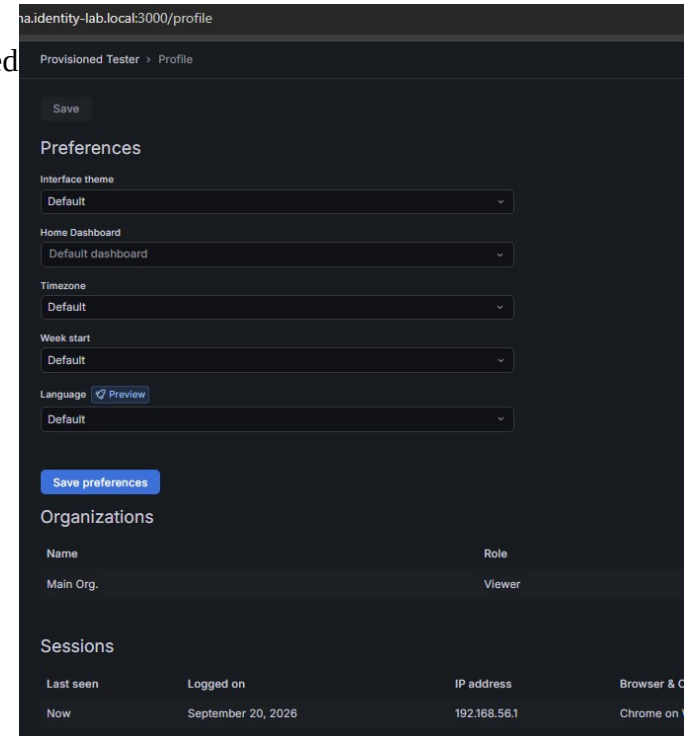
13. Successful Login Through Keycloak

I signed in to Grafana as `provision.test` through Keycloak using the user's domain credentials. After authentication, the browser returned to Grafana with an authenticated user session.



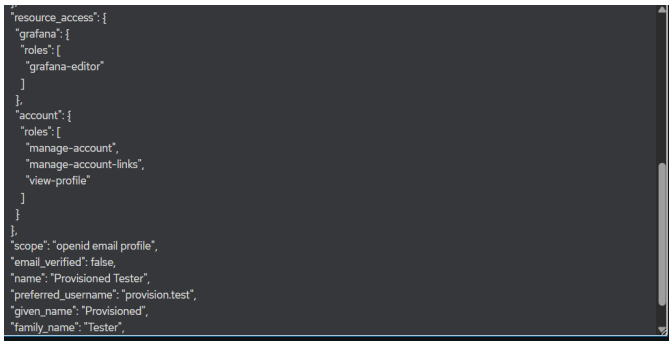
14. Viewer Access Verification

I verified that Grafana assigned the Viewer organization role to `provision.test`. This confirmed that the Keycloak Viewer role was mapped correctly.



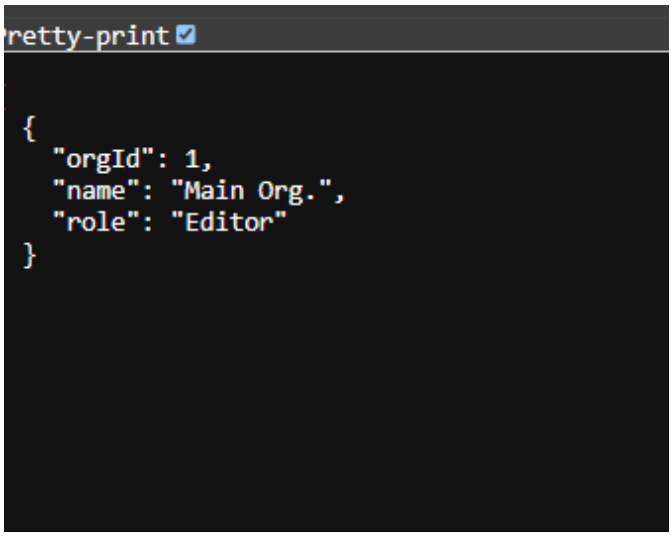
15. Editor Role in the Access Token

I moved `provision.test` from `grafana-viewers` to `grafana-editors` and generated a new token preview. The token contained `grafana-editor` under `resource_access.grafana.roles`.



16. Editor Access Verification

I completed a new OAuth login after changing the user's group membership. Grafana displayed the Editor role, confirming that the updated access level was synchronized.



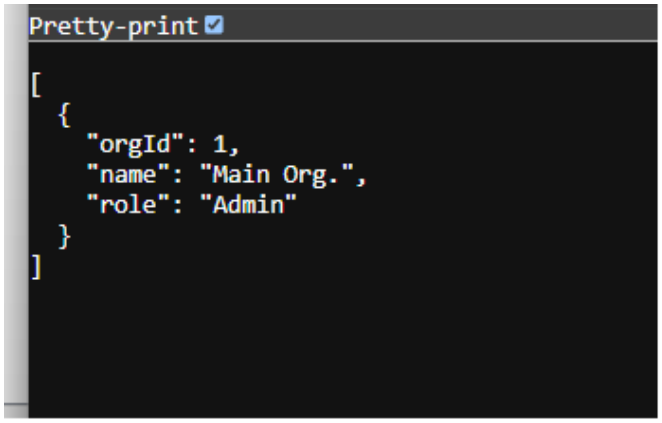
17. Admin Role in the Access Token

I moved `provision.test` from `grafana-editors` to `grafana-admins`. The generated access token contained the `grafana-admin` client role.



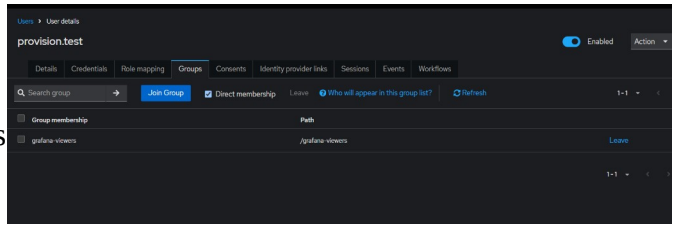
18. Admin Access Verification

I signed in again through Keycloak and verified that Grafana assigned the Admin organization role. This access level provides organization administration without granting global Grafana server administrator privileges.



19. Restoration of Viewer Group Membership

I removed `provision.test` from `grafana-admins` and restored membership in `grafana-viewers`. This returned the user's assigned access to the minimum level required.



20. Final Viewer Access Verification

I completed another OAuth login and confirmed that Grafana displayed the Viewer role again. This verified that the reduced permissions were synchronized from Keycloak.

